

DR: Matching of template template-arguments excludes compatible templates

Document number: P0522R0
Date: 2016-11-11
Project: Programming Language C++, Core Working Group
Reply-to: James Touton <bekenn@gmail.com>
Hubert Tong <hubert.reinterpretcast@gmail.com>

Table of Contents

1. [Table of Contents](#)
2. [Introduction](#)
3. [Overview](#)
4. [Wording](#)
5. [References](#)
6. [Acknowledgments](#)

Introduction

Template *template-parameters* only bind to arguments with matching parameter lists. Matching is currently defined as an exact match; each template parameter in the argument's parameter list must have the same kind and type as the corresponding parameter in the *template-parameter's* parameter list. (There is an ill-advised exception to this rule when a parameter pack appears in the *template-parameter's* parameter list; this paper preserves that special behavior.) The matching rules exclude many reasonable arguments. This paper adjusts the matching rules to invoke partial ordering to determine when a template *template-argument* is a valid match for a *template-parameter*.

This paper resolves Core issue [CWG 150](#).

Overview

This paper allows a template *template-parameter* to bind to a template argument whenever the template parameter is at least as specialized as the template argument. This implies that any template argument list that can legitimately be applied to the template *template-parameter* is also applicable to the argument template.

Example:

```
template <template <int> class> void FI();
template <template <auto> class> void FA();
template <auto> struct SA { /* ... */ };
template <int> struct SI { /* ... */ };
FI<SA>(); // OK; error before this paper
FA<SI>(); // error

template <template <typename> class> void FD();
template <typename, typename = int> struct SD { /* ... */ };
FD<SD>(); // OK; error before this paper (CWG 150)
```

Wording

All modifications are presented relative to N4606.

Modify §14.3.3 [temp.arg.template] paragraph 3:

A template-argument matches a template *template-parameter* *P* when *P* is at least as specialized as the *template-argument* *A*. If *P* contains a parameter pack, then *A* also matches *P* if each of *A*'s template parameters in the *template-parameter-list* of the *template-argument's* corresponding class template or alias template *A* matches the corresponding template parameter in the *template-parameter-list* of *P*. Two template parameters match if they are of the same kind (type, non-type, template), for non-type *template-parameters*, their types are equivalent ([temp.over.link]), and for template *template-parameters*, each of their corresponding *template-parameters* matches, recursively. When *P*'s *template-parameter-list* contains a template parameter pack ([temp.variadic]), the template

parameter pack will match zero or more template parameters or template parameter packs in the *template-parameter-list* of A with the same type and form as the template parameter pack in P (ignoring whether those template parameters are template parameter packs).

[Example:

```
template<class T> class A { /* ... */ };
template<class T, class U = T> class B { /* ... */ };
template<class ... Types> class C { /* ... */ };
template<auto n> class D { /* ... */ };
template<template<class> class P> class X { /* ... */ };
template<template<class ...> class Q> class Y { /* ... */ };
template<template<int> class R> class Z { /* ... */ };
X<A> xa; // OK
X<B> xb; // OK ill-formed: default arguments for the parameters of a template argument are ignored
X<C> xc; // OK ill-formed: a template parameter pack does not match a template parameter
Y<A> ya; // OK
Y<B> yb; // OK
Y<C> yc; // OK
Z<D> zd; // OK
```

—end example]

[Example:

```
template <class T> struct eval;
template <template <class, class...> class TT, class T1, class... Rest>
struct eval<TT<T1, Rest...>> { };
template <class T1> struct A;
template <class T1, class T2> struct B;
template <int N> struct C;
template <class T1, int N> struct D;
template <class T1, class T2, int N = 17> struct E;
eval<A<int>> eA; // OK: matches partial specialization of eval
eval<B<int, float>> eB; // OK: matches partial specialization of eval
eval<C<17>> eC; // error: C does not match TT in partial specialization
eval<D<int, 17>> eD; // error: D does not match TT in partial specialization
eval<E<int, float>> eE; // error: E does not match TT in partial specialization
```

—end example]

Insert a new paragraph after §14.3.3 [temp.arg.template] paragraph 3:

A template *template-parameter* P is at least as specialized as a template *template-argument* A if, given the following rewrite to two function templates, the function template corresponding to P is at least as specialized as the function template corresponding to A according to the partial ordering rules for function templates ([temp.func.order]). Given an invented class template x with the template parameter list of A (including default arguments):

- Each of the two function templates has the same template parameters, respectively, as P or A.
- Each function template has a single function parameter whose type is a specialization of x with template arguments corresponding to the template parameters from the respective function template where, for each template parameter PP in the template parameter list of the function template, a corresponding template argument AA is formed. If PP declares a parameter pack, then AA is the pack expansion PP... ([temp.variadic]); otherwise, AA is the *id-expression* PP.

If the rewrite produces an invalid type, then P is not at least as specialized as A.

References

1. CWG Issue 150: <http://open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4457.html#150>